EE345 Mod3-Lec1: Gradient Descent for Convex Costs

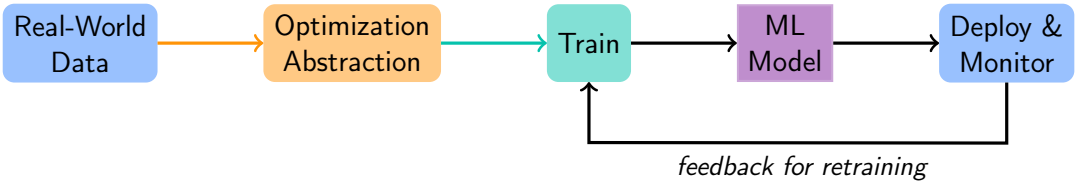
References:

- [OM]: Chapter 3.3, 4, 5 [Optimization Methods]

Overview

1. Module 1: Distances and the Algorithms they Enable
   - Distances and metrics: Euclidean distance, Manhattan distance, cosine distance, worst-case distance, etc.
   - Vectors and matrices: dot product, matrix multiplication, transpose, inverse, etc.
   - Distance based algorithms: k-nearest neighbors (supervised), k-means (unsupervised)
2. Module 2: Linear Models & Regression
   - Convex costs: quadratic functions/forms
   - Linear models and their prevelance in machine learning
   - Least squares: quadratic cost, linear regression
3. Module 3: Gradient Descent, Regularization, Kernel Methods

ML workflow: Role of Linear Algebra & Optimization



This course so far:

- Optimization Abstraction: Data → Optimization Abstraction via losses as norms
- ML models: Distance metric based models (k-NN, k-means); Linear models (least squares regression, attention, PCA)
- Training: Optimizing "convex" objectives (least squares)

Coming up next:

- Training: algorithms such as stochastic gradient descent (bias-variance tradeoff)
- ML models: Linear models: kernel methods
- Optimization Abstraction: Regularization, generalization, overfitting, etc.

ML workflow: Optimization Framework for Learning

Real world task involving data $\longrightarrow$ Optimization Problem $\longrightarrow$ ML model

Canonical Optimization Problem

$$\min_{\theta \in \Theta \subset \mathbb{R}^n} \quad \ell(\theta)$$

Terminology:

- θ: Parameter/optimization or decision variable
- $\ell(\theta)$: Objective/Loss function; in ML will often depend on data Z; write as $\ell(\theta; Z)$.
- $\Theta \subseteq \mathbb{R}^n$: Constraint set

How do we find solutions to optimization problems?

Last Time: From Stability to Regularization

- The minimum-norm solution is the **most stable** interpolant:

$$\theta^\star = \arg\min_\theta \|\theta\|_2^2 \quad \text{s.t.} \quad X\theta = y.$$

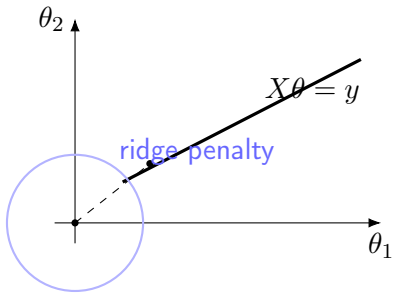
- In practice, data are noisy — exact interpolation can overfit.
- Idea: **trade off fit and simplicity.**

$$\boxed{\min_\theta \ \|X\theta - y\|_2^2 + \lambda\|\theta\|_2^2}$$

- $\lambda > 0$ controls the tradeoff.
- As $\lambda \to 0$, we recover the minimum-norm interpolant.
- As $\lambda \to \infty$, we prioritize simplicity ($\theta \approx 0$).

Last Time: Regularization = Controlled Stability

- Regularization keeps the solution **stable** when y changes slightly.
- It also reduces variance and improves prediction quality.
- Gradient descent with small initialization often behaves as if λ is small but positive (implicit regularization).



Geometrically: regularization picks the point on the affine set $X\theta \approx y$ closest to the smallest-norm ball.

From Least Squares to Optimization

- We solved
$$\min_{\theta} \frac{1}{2}\|y - X\theta\|_2^2.$$
- Closed-form solution: $\hat{\theta} = (X^\top X)^{-1} X^\top y$.
- But for large datasets, computing the inverse is expensive or impossible!
- Idea: **Iteratively improve** θ instead of solving exactly.

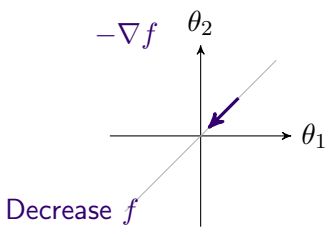
Optimization viewpoint

Find $\theta^\star$ that minimizes a convex cost function $f(\theta)$.

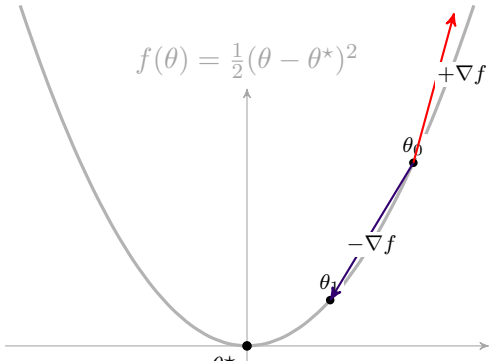
Gradients and the Direction of Steepest Descent

$$\nabla_\theta f(\theta) = \begin{bmatrix} \frac{\partial f}{\partial \theta_1} \\ \vdots \\ \frac{\partial f}{\partial \theta_n} \end{bmatrix}$$

- Points in direction of fastest increase of f.
- So to decrease f, move *opposite* the gradient.

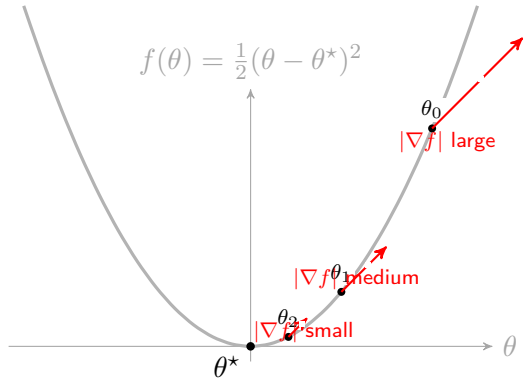


Gradient Descent: Clean Bowl Picture



Move opposite the gradient: $\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$

Gradient Magnitude and Step Size



The gradient points uphill and its magnitude shrinks near $\theta^\star$, so gradient descent takes smaller steps.

Example: Gradient Descent on a Quadratic

$$f(\theta) = \tfrac{1}{2}a\theta^2 - b\theta + c, \qquad \nabla f(\theta) = a\theta - b.$$

- Step-size matters

Key takeaway

Gradient descent finds the minimum of convex functions by iteratively moving opposite the gradient.

Gradient Descent Step Size: Quadratic Example

Dynamical Systems View

Gradient descent on a quadratic in d dimensions:

Spectral radius:

$$\rho(M) := \max_i |\lambda_i(M)| \quad \text{(matrix input only).}$$

Convergence factor (as function of step size): Stability: $r(\eta) < 1$

In words: GD is a linear discrete-time system with update matrix $I - \eta A$. Its eigenvalues must lie inside the unit circle for stability.

Bounding the Error via Matrix Norms

For symmetric A, spectral radius is the largest eigenvalue of $I - \eta A$ in magnitude

$$\|I - \eta A\|_2 = \rho(I - \eta A) = \max_i |1 - \eta \lambda_i(A)|.$$

Hence

Since A is symmetric, eigenvalues describe both the norm and the stability.

Gradient Descent Dynamics in Least Squares

- Objective:

$$f(\theta) = \tfrac{1}{2}\|X\theta - y\|^2, \qquad \nabla f(\theta) = X^\top (X\theta - y).$$

Gradient Descent Dynamics in Least Squares

Unrolling the Gradient Descent Recursion (back-up slide)

- Gradient descent update:

$$\theta_{t+1} = (I - \eta X^\top X)\theta_t + \eta X^\top y, \qquad \theta_0 = 0.$$

- Define

$$A := I - \eta X^\top X, \qquad b := \eta X^\top y.$$

- Then $\theta_{t+1} = A\theta_t + b$.

Iterate (unroll) the recursion

$$\begin{aligned} \theta_1 &= b, \\ \theta_2 &= Ab + b, \\ \theta_3 &= A^2 b + Ab + b, \\ &\vdots \end{aligned}$$

so that $\boxed{\theta_t = \sum_{k=0}^{t-1} A^k b.}$

Unrolling the Gradient Descent Recursion (back-up slide)

Substitute back:

$$\theta_t = \eta \sum_{k=0}^{t-1} (I - \eta X^\top X)^k X^\top y.$$

- This is a matrix geometric series.
- Recall standard geometric series:

$$1 + r + \cdots + r^{t-1} = \frac{1 - r^t}{1 - r}.$$

Using the Matrix Geometric Series

$$\theta_t = \eta \sum_{k=0}^{t-1} (I - \eta X^\top X)^k X^\top y.$$

- Let $B = X^\top X$. The finite geometric series gives

$$\sum_{k=0}^{t-1} (I - \eta B)^k = (\eta B)^{-1} \big[I - (I - \eta B)^t \big].$$

- Substituting:

$$\boxed{\theta_t = (X^\top X)^{-1} \big[I - (I - \eta X^\top X)^t \big] X^\top y.}$$

The tricky part

Finite Steps $\implies$ Implicit Regularization

- That first identity gave us

$$\theta_t = (X^\top X)^{-1}\big[I-(I-\eta X^\top X)^t\big]X^\top y = \eta t B B^{-1}(I+\eta t B)^{-1}X^\top y = \eta t (I+\eta t B)^{-1}X^\top y$$

- Then the tricky part gives us this approximation:

$$\boxed{\theta_t \;\approx\; \left(X^\top X + \frac{1}{\eta t} I\right)^{-1} X^\top y.}$$

- This has the same form as **ridge regression**:

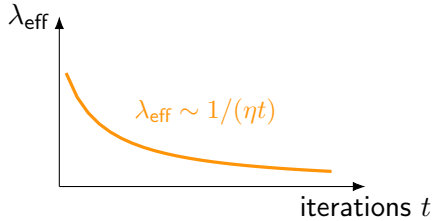
$$\theta_\lambda = (X^\top X + \lambda I)^{-1} X^\top y, \quad \text{with } \lambda_{\text{eff}} \approx \tfrac{1}{\eta t}.$$

Interpretation: "Implicit Regularization"

- Even though we did *not* add an explicit penalty, gradient descent acts as if there were a small ridge term.
- **Early stopping** corresponds to a larger effective λ: smoother, smaller-norm solution.
- **Running to convergence** corresponds to $\lambda_{\text{eff}} \to 0$: recovers the minimum-norm interpolant.

Gradient descent with small initialization behaves like ridge regression with $\lambda_{\text{eff}} \approx \frac{1}{\eta t}$.

Stopping gradient descent early acts like adding an ℓ_2 penalty: controls model complexity without explicit regularization.



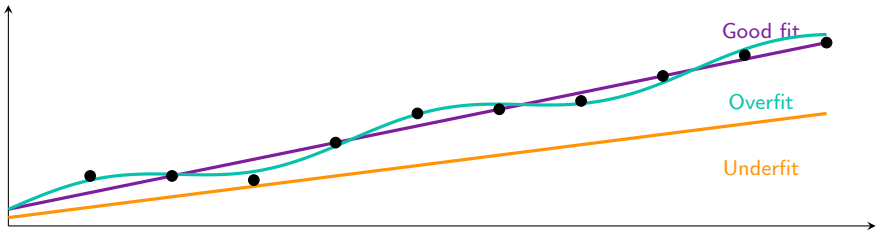
more training $\implies$ less regularization

Why Regularization?

- Our goal is to make predictions that work **beyond the training data**.
- If the model fits the training data too closely, it may fail on new examples — this is called **overfitting**.
- If the model is too simple, it cannot capture patterns — this is **underfitting**.
- Regularization helps us find a **middle ground**.

Underfitting vs Overfitting

- Same training data, three different models:
  - **Underfit (Tangerine):** too simple, misses trend.
  - **Good fit (Purple):** matches overall pattern.
  - **Overfit (Turquoise):** too wiggly, chases every point.



Explicit Regularization

- We can directly control how large the parameter vector is.
- Example: **ridge regression**

$$\min_{\theta} \ \|X\theta - y\|^2 + \lambda\|\theta\|^2$$

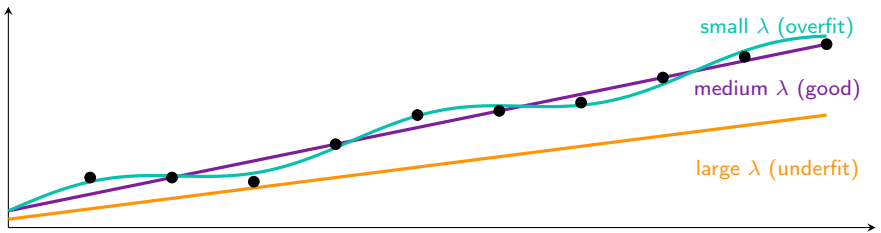
- The second term keeps θ small.
  λ is the **regularization strength**.
  - λ small $\implies$ flexible model.
  - λ large $\implies$ smoother, simpler model.
- Same idea appears with ℓ_1 (lasso) and others. You just get different shapes for the regularization term.

Explicit Regularization (Ridge): Effect of λ

- Ridge regression:

$$\min_{\theta} \|X\theta - y\|^2 + \lambda\|\theta\|^2$$

- λ controls model flexibility:
  - **Large** λ: very simple $\implies$ underfit.
  - **Medium** λ: good balance.
  - **Small** λ: very flexible $\implies$ overfit.

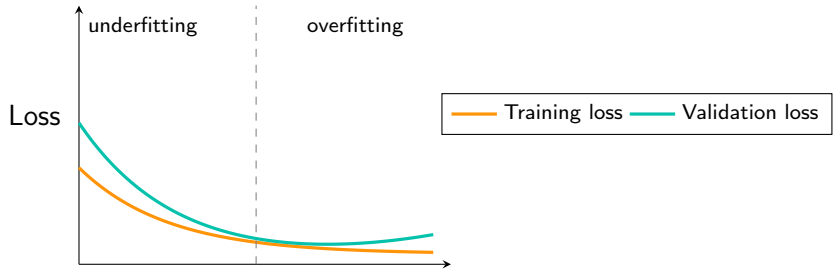


Implicit Regularization: From the Algorithm Itself

- Sometimes we do **not** add any penalty, but the training process still prefers "simpler" models.
- Examples:
  - Gradient descent on $\min_\theta \|X\theta - y\|^2$ (when there are many solutions) picks the one with the **smallest norm**.
  - Stopping gradient descent **early** often gives smoother, more stable predictions.
- These effects come from how the algorithm moves and stops, not from an explicit penalty term.

Effect of Regularization on Training and Validation Loss

- Strong regularization (λ large): model too simple $\Rightarrow$ underfitting.
- Weak regularization (λ small): model too flexible $\Rightarrow$ overfitting.
- The sweet spot minimizes validation loss.



Early Stopping Acts Like Regularization

- Each gradient step changes θ by $-\eta \nabla_\theta \|X\theta - y\|^2$.
- If we stop after only a few steps, θ is still close to 0.
- This behaves like shrinking θ with an ℓ_2 penalty.
- So early stopping is a kind of **implicit regularization**.
- In practice:
  - Track training and validation loss.
  - Stop when validation loss starts increasing.

Early Stopping as Implicit Regularization

- As we train (steps $t = 0, 1, 2, \dots$):
  - **Training loss** keeps decreasing.
  - **Validation loss** decreases at first, then increases once we overfit.
- Stop at t^* where validation loss is smallest — this is **early stopping**.
- Effect: prevents overly complex parameters $\implies$ behaves like regularization.